Hyperparameter Tuning and Hallucination Analysis in a RAG-Based Video Game Lore Chatbot

Anthony Salva Mijares
Department of Computer Studies & Engineering (CSE)
José Rizal University
Mandaluyong, Philippines
anthony.mijares@my.jru.edu

***Abstract*—Retrieval-Augmented Generation (RAG) limits language model hallucinations by grounding generated answers in specific external documents. This evaluation tested a domain-specific chatbot built on The Legend of Zelda Encyclopedia across fourteen distinct hyperparameter configurations. The pipeline utilized a Chroma vector database and the openai/gpt-oss-20b language model. Adjustments to chunk size, overlap, Top-K retrieval depth, and temperature were measured against broad summaries, specific fact extractions, and out-of-domain queries regarding games released after the original encyclopedia was published. Without strict fallback rules and low temperatures, the language model produced severe hallucinations instead of admitting a lack of information. Furthermore, providing excessive context volume introduced background noise that triggered false negative refusals rather than improving retrieval accuracy. Extracting exact factual details requires tighter retrieval parameters than generating broad summaries. The calibrated pipeline successfully prevents ungrounded responses and was deployed as a secure, interactive web application using the Streamlit framework with strict API key protection.**

Keywords—hallucination mitigation, hyperparameter tuning, large language models, retrieval-augmented generation, vector databases

I. SYSTEM ARCHITECTURE

The chatbot was built using a Retrieval-Augmented Generation (RAG) framework. RAG is an architecture that forces the language model to read specific external documents before generating an answer. This ensures that the generated responses are based purely on factual documents rather than the model's unpredictable built-in memory [1]. The pipeline is divided into four main steps.

A. Document Loading

The retrieval pipeline requires a reliable source of information. For this experiment, the source document was The Legend of Zelda Encyclopedia, which contains official game lore and development notes. The dataset was obtained directly from Archive.org as a 257 kilobytes plain text file. Using this text format instead of the original 37 megabytes PDF publication is better for LLMOps because it avoids unnecessary memory consumption and image rendering errors. LangChain's DirectoryLoader was used to read the file into the notebook [2].

The exact coverage of the encyclopedia is important to understand before reading the experimental results. The encyclopedia covers every canonical Zelda game installment from the original 1986 Legend of Zelda up to Tri Force Heroes, which was released in 2015 [3]. The book itself was published in 2018 by Dark Horse Comics in collaboration with Nintendo. Because the publication was already finalized before Breath of the Wild launched in March 2017, that title and all games that followed it, including Tears of the Kingdom released in 2023, are completely absent from the text. This natural gap in coverage is significant because it creates a clean boundary for testing. Any question about Hyrule, the Triforce, or game mechanics documented before 2017 falls inside the knowledge base. Any question about Tears of the Kingdom, the Zonai civilization, or Ultrahand abilities falls completely outside of it. The out-of-domain test queries for this experiment were designed around this exact boundary.

B. Text Chunking Strategy

Language models have strict memory limits, known as context windows [4]. Because of this, they cannot read a massive book all at once. To solve this problem, a LangChain RecursiveCharacterTextSplitter was used to cut the large document into smaller pieces called chunks. The splitter prioritizes paragraph breaks, line returns, and spaces so it does not awkwardly cut words in half.

The text was cut into 500-character chunks with a 50-character overlap. The overlap creates a protective buffer between the chunks, ensuring that sentence continuity is preserved and proper nouns are not accidentally cut in half across chunk boundaries, resulting in 556 chunks as demonstrated in Fig. 1.

```
from langchain_community.document_loaders import DirectoryLoader
100%|██████████| 1/1 [00:36<00:00, 36.02s/it]Loaded 1 raw document(s) and split into 556 chunks.
```

Fig. 1. Document Ingestion and Chunking.

C. Vector Embeddings and Database

Language models cannot perform direct mathematical similarity searches across raw text files. In order to allow the retrieval pipeline to search for information quickly, the text chunks had to be converted into high-dimensional numerical vectors [5]. This conversion was done locally using the open-

source all-MiniLM-L6-v2 embedding model, which avoids cloud computing costs.

Once the chunks were converted into vectors, they were indexed into a Chroma vector database [6]. When a user asks a question, the pipeline converts the question into a vector and compares the vector direction against all stored chunks to find the closest match, as shown in Fig. 2.

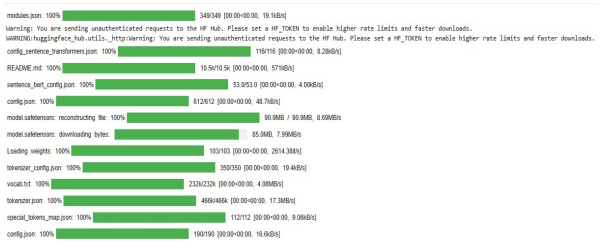


Fig. 2. Vector Database Initialization.

D. Language Model Integration

The final step connects the vector database to the language model. The model that was initially used is llama-3.1-8b-instant, however that specific model was decommissioned on the Groq platform, which resulted in a strict HTTP 404 model_not_found runtime error. In order to fix this issue, the code was updated to use openai/gpt-oss-20b instead. The complete pipeline takes the user's question, retrieves the matching text chunks from Chroma, places them into the prompt, and forwards the data in a single automated call.

II. Implementation Details

The RAG pipeline was evaluated by adjusting its hyperparameters across fourteen distinct test cases. Each case changed a specific combination of chunk size, overlap, Top-K retrieval depth, temperature, and the presence or absence of a fallback rule. The complete set of configurations is listed in Table I.

TABLE I. Pipeline Hyperparameter Configurations Across Test Cases

Case	Hyperparameters					
	Focus	***Chunk Size***	***Chunk Overlap***	***Top-K***	***Temp***	***Fallback Rule***
1	Baseline In-Domain	500	50	3	0.0	Active
2	Baseline Out-of-Domain	500	50	3	0.0	Active
3	Hallucination Stress-Test	500	50	3	1.0	Removed
4	Max Embedding Capacity	1000	100	3	0.0	Active
5	Increased Overlap	500	100	3	0.0	Active
6	Expanded Retrieval	500	50	5	0.0	Active
7	Minimal Retrieval	500	50	1	0.0	Active
8	Micro-Chunking	250	25	3	0.0	Active
9	Elevated Temperature	500	50	3	0.5	Active
10	Zero Overlap	500	0	3	0.0	Active
11	Expanded and Zero Temp	1000	100	5	0.0	Active
12	Expanded and Slight Temp	1000	100	5	0.2	Active
13	Focused Retrieval	1000	100	3	0.2	Active
14	Calibrated Retrieval	1000	100	4	0.2	Active

The Chunk Size hyperparameter determines how many characters each piece of text contains after the document is split. A smaller chunk size, such as 250 characters, produces more chunks but each one carries very little context. A larger chunk size, such as 1000 characters, produces fewer chunks but each one carries more surrounding information. The Chunk Overlap parameter controls how many characters are shared between two adjacent chunks. An overlap of 0 means each chunk is completely independent, which risks cutting a sentence in half at a split point. An overlap of 100 characters ensures that the end of one chunk and the start of the next share enough text to preserve sentence continuity.

The Top-K hyperparameter controls how many chunks of text the database sends to the language model. A value that is too low, such as K=1, limits the retrieved context so much that the model lacks enough information to answer accurately. A value that is too high, such as K=5, overloads the model with irrelevant background text that buries the specific answer it needs.

The Temperature hyperparameter controls how creative the language model is. Setting the temperature to 0.0 enables greedy decoding, forcing the model to strictly choose the most mathematically probable token. This makes the answers factual and grounded. If the temperature is increased to higher values like 1.0, the probability distribution flattens, causing the model to sample more varied words. This creativity directly increases the risk of hallucinations [7].

III. Experimental Results & Hallucination Analysis

In order to evaluate the chatbot, three categories of test queries were designed to cover different retrieval scenarios. A broad summary question was used to test whether the chatbot could pull and synthesize information from multiple retrieved chunks into a coherent response. An out-of-domain question was used to test whether the fallback rule correctly prevented the model from answering when no relevant evidence existed in the knowledge base. A specific factual question was used to test whether the Top-K retrieval depth was precise enough to locate a single exact answer buried inside the encyclopedia. The test queries and the actual answers generated by the model are recorded in Table II.

TABLE II. Benchmark Test Queries and Baseline Responses

Category	Query Details and Model Evaluation		
	Test Query	***Expected Response***	***Actual Output***
In-Domain	"Summarize the key information found in the documents."	A summary grounded in Zelda lore.	An accurate summary covering game development notes and maps.

Category	Query Details and Model Evaluation		
	Test Query	***Expected Response***	***Actual Output***
Out-of-Domain	"Explain the origins of the Zonai civilization and their technological devices in Tears of the Kingdom."	Refusal to answer due to missing evidence.	"I cannot answer based on the provided domain data."
Specific Fact	"Where does the Master Sword rest?"	Extraction of the exact location.	"It rests in a pedestal in the castle."

As shown in Table III, the chatbot's outputs and grounding status are detailed across all fourteen test configurations, showing which parameter combinations remained factual and which degraded into hallucinations or errors. Performance was evaluated across broad summaries, specific fact extractions, and out-of-domain refusals to identify how adjustments to chunk size, retrieval depth, and temperature trigger distinct failure modes.

TABLE III. EXPERIMENTAL RESULTS ACROSS ALL TEST CASES

Case	Model Outputs and Hallucination Assessment			
	Broad Domain Output	***Specific Fact Output***	***Out-of-Domain Output***	***Hallucination Status***
1	Accurate summary of development notes	N/A	N/A	Grounded
2	N/A	N/A	Exact fallback refusal	Grounded
3	N/A	N/A	Fabricated essay	Severe Hallucination
4	High-coherence summary	N/A	Fallback refusal	Grounded
5	Accurate summary of development materials	N/A	Fallback refusal	Grounded
6	Detailed summary covering games	N/A	Fallback refusal	Grounded
7	Isolated fragments only	N/A	Fallback refusal	Incomplete
8	Fragmented terms	N/A	Fallback refusal	Fragmented
9	Stylistically diverse summary	N/A	Fallback refusal	Grounded
10	Factual summary missing context	N/A	Fallback refusal	Grounded
11	Comprehensive summary	"The Master Sword rests inside the Sealed Temple." (incorrect)	Fallback refusal	Minor Hallucination
12	Comprehensive summary	Fallback refusal (false negative)	Fallback refusal	False Negative
13	Accurate summary	"It rests on a pedestal." (partial: no castle location)	Fallback refusal	Partial
14	Complete summary	"It rests in a pedestal in the castle."	Fallback refusal	Optimal

A. Hallucination Stress-Test: Baseline vs. Ungrounded Comparison

Language models are trained to generate complete and fluent responses [8]. As a result, they tend to produce an answer even when they lack supporting evidence. The baseline configuration in Case 1 established standard operational behavior using a chunk size of 500, an overlap of 50, Top-K of 3, and a temperature of 0.0. When queried for a broad summary of the documents, the model returned an accurate response grounded strictly in official development notes, proving that the pipeline retrieves and synthesizes relevant context properly under normal conditions.

Evaluating this baseline against out-of-domain queries revealed the necessity of strict prompt rules, since the encyclopedia contains no information regarding later titles like Tears of the Kingdom. Maintaining this exact baseline configuration, Case 2 tested the model against an out-of-domain question regarding the Zonai civilization, where the active fallback rule correctly output 'I cannot answer based on the provided domain data.' Removing the fallback rule and raising the temperature to 1.0 in Case 3 produced the opposite outcome under the same query. Rather than refusing to answer, the model generated an extensive, ungrounded breakdown of the civilization and its technology. The comparison demonstrates that without prompt restrictions and low temperatures, language models will produce responses without document grounding rather than admitting a lack of information, as illustrated in Fig. 3.



Fig. 3. Hallucination Stress-Test Comparison.

B. Evaluation of Single Hyperparameter Changes

Following the baseline tests, the next phase of the experiment evaluated how sensitive the chatbot was to individual parameter changes. Cases 4 through 10 modified

one setting at a time while keeping all other variables constant. This controlled approach made it possible to see the exact impact of each adjustment on retrieval accuracy and answer quality.

Doubling the chunk size to 1000 characters in Case 4 allowed each chunk to hold more text. With three chunks providing up to 3000 characters of continuous reading, the model generated a clear and complete summary because sentences were not cut off in the middle. For Case 5, doubling the overlap to 100 characters created 580 chunks with more shared text between them. This safety buffer kept character names and specific game terms intact across split points, helping the model return an accurate summary with smooth sentences. Raising Top-K from 3 to 5 in Case 6 increased the total retrieved text from 1500 to 2500 characters. Giving the model five chunks allowed it to pull information from more game titles, although having more text also added some background noise to the answer. Lowering Top-K to 1 in Case 7 caused the opposite problem, where the model did not receive enough text to work with. Limited to just a single 500-character chunk, it could not write a proper summary and only output broken characters and cut-off titles like 'ATATAT...' and '18 I The Leg'.

A similar issue happened in Case 8 when cutting the chunk size down to 250 characters. This split the document into 1206 tiny chunks. Because each piece only held short phrases, the model could not connect the ideas together and only output disconnected bullet points. Raising the temperature to 0.5 in Case 9 tested how the model writes when it has more freedom to choose words. The answer used more varied vocabulary, but the active fallback rule successfully stopped the model from inventing false claims. Finally, removing the overlap completely in Case 10 created hard split points across 545 chunks. Without shared text between chunks, sentences were cut in half, leading to missing information and broken transitions between development notes.

C. *Analysis of Specific Fact Extraction*

The final phase of testing shifted focus from broad domain comprehension to the extraction of a single fact. This evaluation introduced the specific domain query 'Where does the Master Sword rest?'. The source document establishes that the weapon rests in a pedestal in the castle, but setting Top-K to 5 in Case 11 supplied 5000 characters of context across five large chunks. This excessive volume created too much background noise and distracted the model, producing the incorrect answer 'The Master Sword rests inside the Sealed Temple.' Keeping Top-K at 5 while raising the temperature slightly to 0.2 in Case 12 caused a false negative refusal where the chatbot unnecessarily triggered the fallback rule and claimed it lacked enough data to answer.

Reducing Top-K to 3 in Case 13 narrowed the retrieval pool excessively and led to an incomplete response. The chatbot returned the text 'It rests on a pedestal.' Providing a moderate context volume through a Top-K of 4 and a temperature of 0.2 in Case 14 proved optimal for exact fact extraction. This configuration balanced the retrieved text perfectly, allowing the model to produce the exact match 'It rests in a pedestal in the castle.' Therefore, extracting specific details requires tighter retrieval parameters than generating broad summaries.

IV. Conclusion

Developing an effective domain-specific RAG pipeline extends beyond simply connecting a language model to a vector database. The results across fourteen test configurations showed that specific development decisions directly determined whether the chatbot produced accurate answers or hallucinated responses. System prompt guardrails proved to be mandatory, as an explicit fallback rule gives the model an operational boundary that prevents generating ungrounded claims when document evidence is missing [9]. Retrieval parameters also required careful calibration, since excessive context windows introduced noise that triggered false negative refusals rather than improving accuracy [10]. Balancing the Top-K parameter and chunk boundaries ensured that specific factual details could be extracted without overwhelming the model with irrelevant background text.

In addition to retrieval tuning, production pipelines require strict API-key security practices. Hardcoding credentials directly within a public notebook exposes the key to unauthorized access and credential theft. Storing tokens within secure environment variables using the Python getpass module prevents malicious exploitation and protects API consumption limits [11]. Beyond the experimental configurations, the verified pipeline was packaged and deployed as a browser-accessible web application called Hyrule Chronicles AI using the Streamlit framework. This deployment demonstrates that the calibrated RAG architecture can successfully transition from experimental testing into a practical, interactive tool for end users without requiring them to run code directly.

References

[1] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, Dec. 2023. doi: 10.48550/arXiv.2312.10997.

[2] LangChain, "LangChain Documentation: Document loaders, Text splitters, and Chains," LangChain Inc., 2024. [Online]. Available: https://python.langchain.com/docs/modules/

[3] Nintendo and M. Dyer, The Legend of Zelda Encyclopedia, Dark Horse Comics, 2018.

[4] N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," Transactions of the Association for Computational Linguistics, vol. 12, 2024. doi: 10.1162/tacl_a_00638.

[5] L. Wang et al., "Text Embeddings by Weakly-Supervised Contrastive Pre-training," arXiv preprint arXiv:2212.03533, Dec. 2022. doi: 10.48550/arXiv.2212.03533.

[6] Chroma, "Chroma: The AI-native open-source embedding database," ChromaDB, 2024. [Online]. Available: https://docs.trychroma.com/

[7] Y. Zhang et al., "Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models," arXiv preprint arXiv:2309.01219, Sep. 2023. doi: 10.48550/arXiv.2309.01219.

[8] J. Lin et al., "TruthfulQA: Measuring How Models Mimic Human Falsehoods," in Proc. 60th Annual Meeting of the Association for Computational Linguistics (ACL), 2022, pp. 3214-3252. doi: 10.18653/v1/2022.acl-long.229

[9] S. Dhuliawala et al., "Chain-of-Verification Reduces Hallucination in Large Language Models," arXiv preprint arXiv:2309.11495, Sep. 2023. doi: 10.48550/arXiv.2309.11495

[10] J. Chen et al., "Benchmarking Large Language Models in Retrieval-Augmented Generation," arXiv preprint arXiv:2309.01431, Sep. 2023. doi: 10.48550/arXiv.2309.01431

[11] Python Software Foundation, "getpass - Portable password input," Python 3.10.13 Documentation, 2023. [Online]. Available: https://docs.python.org/3/library/getpass.html